



AgroMaster

Technical Overview

How the system works, and why it was built this way

VERSION	1.0
DATE	16 September 2026
AUTHOR	Minidu Dhananjana
LIVE	https://agromaster.duckdns.org
REPOSITORY	github.com/minidu10/cultivation-help-app

1. What this system is

AgroMaster helps a small-scale Sri Lankan farmer answer one question that is surprisingly hard to answer without records: **did this crop actually make money?**

A farmer plants a crop, spends money on it over several months — seed, labour, fertiliser, irrigation, transport — and eventually sells the harvest. The costs are spread over time and paid in cash; the income arrives in one or two lumps. Without writing it down, "was the paddy worth it?" becomes guesswork.

The app records each crop from planting to harvest, every cost against it, and every sale. From that it calculates profit per crop, and layers three assistive features on top: an AI advisor that answers farming questions, live weather for the farm's location, and email reminders for time-sensitive tasks.

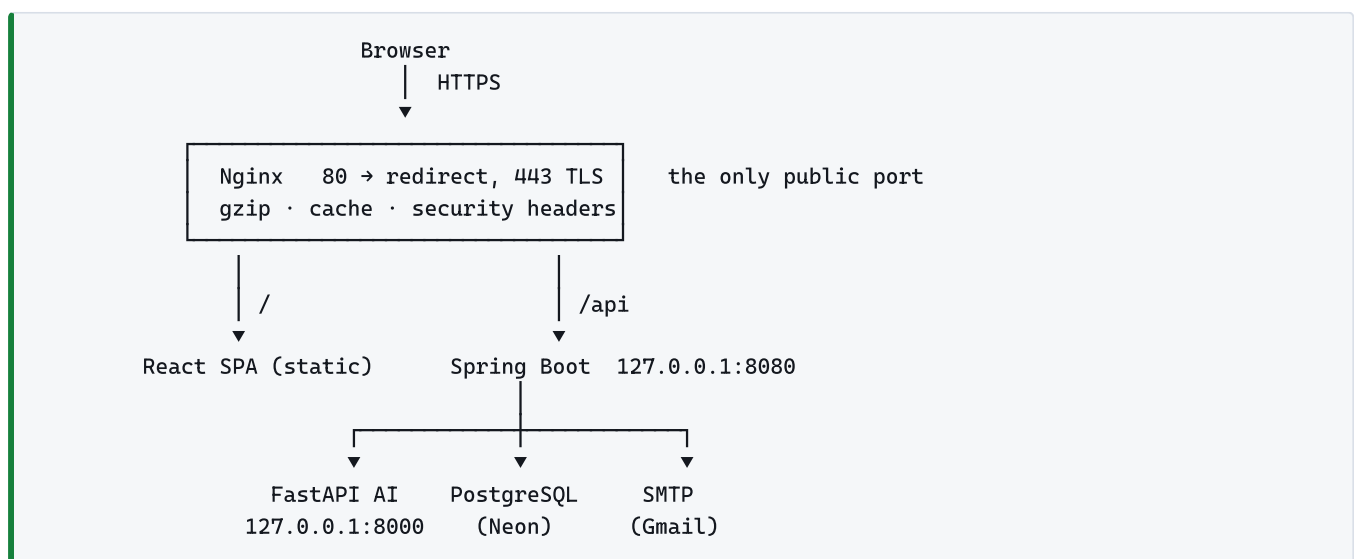
It is **single-farmer per account**. No organisations, no shared farms, no roles. Every record belongs to exactly one user, and that ownership is enforced on every database query rather than only in the interface.

Size

	FILES	LINES
Backend (Java)	72	~4,470
Frontend (React)	28	~4,370
AI service (Python)	3	~258

The AI service is deliberately tiny. Almost all of its job is turning a farming question into an API call and returning plain text.

2. Architecture



Following one request

A farmer opens the dashboard and the browser asks for `/api/crops` :

1. **Nginx** receives it on 443, terminates TLS, and sees the path starts with `/api` , so it proxies to the backend rather than serving a file.
2. **Spring Security** runs `JwtFilter` before anything else. It reads the `Authorization: Bearer` header, verifies the token's signature, and loads the user. No token, or a bad one, and the request stops here with a 403.
3. **CropController** receives the authenticated user object.
4. **CropService** queries `findByIdOrderByCreatedAtDesc(user.getId())` — note the user id is part of the query, not a filter applied afterwards.
5. **PostgreSQL** (hosted on Neon, outside AWS) returns the rows.
6. The response travels back out through nginx, gzipped.

Why each hop exists

Nginx in front — it terminates TLS, serves the React build as static files with compression and cache headers, and is the single public entry point. The backend and AI service bind to `127.0.0.1` and are unreachable from the internet.

A separate AI service — the AI work is Python (the OpenAI SDK, prompt handling), while the business logic is Java. Keeping them apart means the AI provider can be swapped without touching the backend, and a failing AI provider cannot take down crop management.

The database outside AWS — Neon holds the data, so the EC2 instance is disposable. It can be stopped, destroyed or rebuilt without touching a single row. This matters a great deal for a machine that is powered off between demos.

3. The three services

Backend — Spring Boot 4, Java 17

Owns everything that matters: authentication, the domain model, ownership rules, the database, and outbound email. It also acts as the gateway to the AI service.

Layered `controller` → `service` → `repository` → `entity` , with DTOs at the boundary so JPA entities are never serialised directly to the browser.

Frontend — React 19, Vite, Recharts

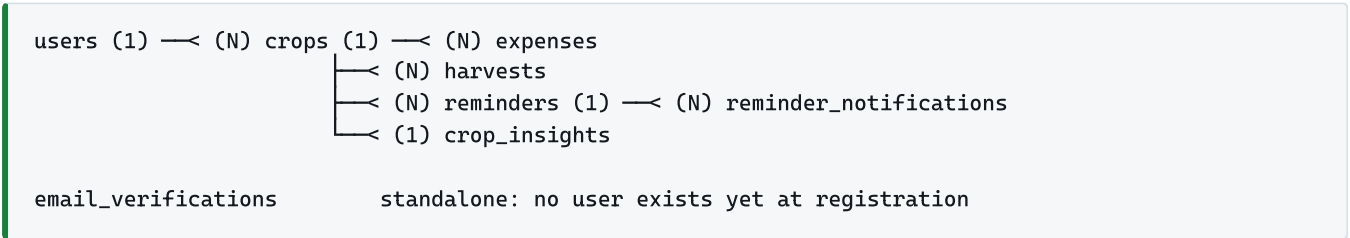
A single-page application. It holds no secrets: the API is reached over relative paths (`/api/...`), which nginx proxies in production and the Vite dev server proxies locally. The same build therefore runs in both places with **no environment file at all** — even the Google client id is fetched at runtime from `GET /api/auth/config` .

Routes are lazy-loaded, so the landing page does not carry the dashboard's charting library.

AI service — FastAPI, Python 3.12

Three endpoints: `/ai/ask` , `/ai/insights` , `/ai/health` . It talks to one OpenAI-compatible API and returns plain text. It has no database, no authentication of its own, and is not reachable from the internet — the backend calls it over the internal Docker network.

4. Data model



Eight tables, built by eleven Flyway migrations (V0–V10).

Why it is shaped this way

Everything hangs off `crops` . An expense without a crop is meaningless, so `crop_id` is mandatory and every child table cascades on delete. Deleting a crop removes its costs, sales and reminders in one operation, with no orphans.

`email_verifications` **has no `user_id`** — and that is the interesting part. Verification happens *before* the account exists, so there is no user to reference. More on why in section 5.

`reminder_notifications` **exists rather than a flag on the reminder**. Each reminder is announced twice (two days ahead, then one day ahead), which a single timestamp column cannot express.

Schema changes only through Flyway. Hibernate's `ddl-auto` is `none` , so the schema is never altered implicitly. Migrations run automatically at startup — which is why pointing the app at an empty Neon database built the entire schema on first boot with no manual step.

Indexes

PostgreSQL does not index foreign keys automatically, and the original schema had no indexes at all. Every repository query filtered on an unindexed column, so every one was a sequential scan. V4 added composite indexes ordered to satisfy both the filter and the sort each query already asks for:

INDEX	SERVES
<code>idx_crops_user_created</code>	crop list for a user, newest first
<code>idx_expenses_crop_date</code>	expenses per crop, and the expense total
<code>idx_harvests_crop_date</code>	harvests per crop, and the revenue total
<code>idx_reminders_due</code>	partial index, pending reminders only

Confirmed afterwards with `EXPLAIN ANALYZE` : `Index Scan` , not `Seq Scan` .

5. Authentication

Three ways in: email and password with verification, password reset, and Google.

Registration — three steps

```
POST /api/auth/send-code    { email }          → 6-digit code emailed
POST /api/auth/verify-code { email, code }     → confirmed
POST /api/auth/register    { name, email, ... } → account created, JWT returned
```

Verification happens before the account is created, not after. The alternative — create the user, mark them unverified, block login — means a mail delivery failure locks somebody out of an account they already own, which then needs a resend endpoint and a support path. Verifying first means the failure mode is simply "cannot register yet", recoverable by retrying, with no orphaned rows.

Why codes rather than emailed links

A code can be typed on a different device from the one reading the email, which matters where a phone is shared. The trade-off is that **a 6-digit code is only a million possibilities** and is therefore brute-forceable. Four rules make it safe, and all four are required:

RULE	VALUE
Expiry	10 minutes
Maximum attempts	5, then the code is dead
Resend cooldown	60 seconds per address
Single use	consumed at registration

The code itself is never stored — only `SHA-256(email + ":" + code)`, so a database leak does not hand over live codes.

Google sign-in

The browser obtains an ID token from Google; the backend verifies its **signature, issuer, audience and expiry** against Google's published keys. This is the security-critical part: without verification, anyone could post a hand-written JSON blob naming any email address and be logged in as that user. Tested with a forged `alg: none` token — rejected with 401.

Signing in with an address that already has a password account **links the two**, but only when Google reports the address as verified. Otherwise an unverified Google account could claim someone else's email.

A Google-only account has no password at all. The login path checks for this explicitly — without that check it would throw a `NullPointerException` inside the password encoder.

Tokens

Stateless JWT, 24-hour expiry. `users.password_changed_at` is stamped on every password change, and `JwtFilter` rejects any token issued before it. Without that, resetting a password would not evict someone holding a stolen token — the one action a worried user knows to take would do nothing.

Rate limiting

Per-IP limits on `/api/auth/**`, because the per-email cooldown does not stop an attacker who simply rotates addresses:

ENDPOINT	REQUESTS/MINUTE
<code>send-code</code> , <code>forgot-password</code> , <code>register</code>	5
<code>login</code> , <code>verify-code</code> , <code>reset-password</code>	10

Without this, `send-code` can be driven in a loop against arbitrary third-party addresses — a mail bomb sent from this project's domain, which would get the sender flagged as spam and exhaust the daily SMTP quota.

6. The AI layer

One client, not one per provider

Gemini, Groq, OpenAI, OpenRouter and Ollama all expose the same chat-completions protocol. The service therefore uses **one OpenAI SDK client** with a configurable base URL:

```
AI_API_KEY    = <key>
AI_BASE_URL   = https://api.groq.com/openai/v1
AI_MODEL      = openai/gpt-oss-120b
```

Switching providers is two lines of configuration and no code change.

This was not a theoretical preference. The service originally had three separate integrations, and adding a second SDK pulled in a newer `httpx` that broke the first one **at import time** — taking down the whole service, including the provider that was actually in use. Three SDKs meant three dependency trees able to break each other.

Dependencies went from seven to four.

Insight caching

The dashboard shows an AI assessment per crop. Originally it asked the model on every page load — four crops meant four calls each time, producing identical text because nothing had changed.

Now each insight is stored against a **fingerprint**: a SHA-256 of the exact figures it was derived from (crop name, status, total expenses, total revenue). While that fingerprint is unchanged the stored text is still correct, so it is served from the database. Add an expense and the fingerprint moves, so exactly that one crop regenerates.

Measured: eight requests produced five model calls, each triggered by a real data change. Opening the dashboard fifty times now costs nothing.

Output shape

The model is asked for **plain text, no markdown, under 110 words**, because the interface renders text rather than markdown — asterisks would otherwise appear literally on screen. A server-side stripper is a safety net for when the model formats anyway, written carefully enough to leave `Rs. 5000 * 2 acres` alone.

One subtlety worth knowing: reasoning models spend part of their token budget on hidden thinking that is never returned. With `max_tokens: 1024` the visible answer was being cut off mid-sentence at 77 tokens. Raising the ceiling to 4,096 fixed it — `max_tokens` is a limit, not a reservation, and brevity comes from the prompt instead.

7. Reminders

A farmer sets reminders against a crop — fertilise, irrigate, scout for pests. Two design decisions shape how they are delivered.

Notice is given in advance, two days ahead and again one day ahead, rather than at the moment a task falls due. A reminder to buy fertiliser is useless the minute it is needed; it is useful the day before.

Everything for one farmer on one day goes in a single email. Sending one message per reminder floods the inbox during a busy week and trains people to ignore the mail entirely. The job runs once daily at 07:00, so more than one message per farmer per day is impossible by construction.

```
4 farm tasks coming up

TOMORROW (Mon 14 Sep)
- Check drip emitters - Tomato at 6:30 AM
- Whitefly scouting - Tomato at 4:00 PM

DAY AFTER TOMORROW (Tue 15 Sep)
- Apply urea top dressing - Tomato at 7:00 AM
- Order pesticide stock - Tomato at 3:30 PM
```

Duplicate sends are prevented by the database, not by job logic. `reminder_notifications` has a unique constraint on `(reminder_id, lead_days)`, so a restart, an overlapping run, or a second instance hits a constraint violation rather than emailing twice. Rows are written only *after* a successful send, so a mail failure retries tomorrow instead of being silently lost.

Farmers switch the emails off in Settings, and the opt-out is applied in the SQL query — an opted-out farmer is never even a candidate.

Timezone matters here more than anywhere else. Reminders are stored as zoneless local times and containers default to UTC, so without `TZ=Asia/Colombo` a 07:00 reminder would fire at 01:30.

8. Security

CONTROL	WHAT IT PREVENTS
BCrypt password hashing	Password recovery from a database leak
Codes stored as SHA-256	Leaked codes being usable
10-min expiry, 5 attempts, single use	Brute-forcing a 6-digit code
Per-IP rate limits on auth	Mail bombing and password guessing
Identical response for unknown emails	Discovering which addresses are registered
Google token signature verification	Forged tokens impersonating any user
<code>password_changed_at</code> check	A stolen token surviving a password reset
Ownership in every query	Reading another farmer's crops by changing an id
API bound to <code>127.0.0.1</code>	The API being reachable over plain HTTP, bypassing TLS
AI behind authentication	Anyone spending this project's API credit
Swagger disabled in production	Publishing a complete map of the API
No JWT secret fallback	Booting with a publicly known signing key
HSTS, X-Frame-Options, CSP	Downgrade attacks and clickjacking
fail2ban	Brute-force noise against the one open port

Two of these are worth expanding.

The AI endpoint was the most serious issue found. It was originally proxied publicly by nginx with no authentication — an open, unmetered LLM endpoint billed to this project's API key. A trivial script could have drained the quota. It now sits on the internal network with the authenticated backend in front of it.

Login previously leaked account existence by returning "User not found" versus "Wrong password". Both now return the same message. This matters more once a password-reset flow exists, since the pair together let someone enumerate users.

9. Deployment and CI/CD

On `git push` to main

```
backend → mvnw verify      ] every push and pull request
frontend → npm ci, eslint, build ]
images   → build 3, push to ghcr.io ] main only
deploy   → ssh, pull, restart
```


Images are built in CI, never on the server. The 1 GB instance cannot reliably compile Java — the Maven build was frequently OOM-killed, which is the reason a swapfile existed at all. GitHub Actions builds and publishes; the server only pulls. Deploys went from 3–5 minutes with a failure risk to about 30 seconds, and the instance has no JDK, Maven or Node installed.

Every image is tagged `:latest` **and** `:<commit-sha>`, so rolling back is setting `IMAGE_TAG` to an earlier commit and restarting.

On boot

The instance is powered off between demos, and AWS assigns a new public IP on every start. A systemd unit runs, in this order:

- 1. Tell DuckDNS the new IP
- 2. `docker compose pull` and start
- 3. Attempt certificate renewal, reload nginx if it changed

Renewal comes last because the HTTP-01 challenge needs nginx already serving port 80. An expired certificate does not prevent nginx from starting — it serves an invalid one — so even after months powered off the instance comes up and then repairs its own certificate.

One compose file

`COMPOSE_PROFILES` decides which services start, so the command is identical locally and in production and the two configurations cannot drift.

SERVICE	LOCAL	PROD
Postgres container	starts	not started (Neon)
Mailpit mail catcher	starts	not started (Gmail)
backend, ai-service	starts	starts
nginx frontend	not started	starts

Cost

Free-tier EC2, Neon free tier, DuckDNS, GHCR — effectively zero. **No Elastic IP**, deliberately: AWS bills for one attached to a stopped instance, which would be most of the time.

10. Running it

Locally

```
cp .env.example .env
docker compose up -d --build
cd frontend-web && npm install && npm run dev
```

Frontend	http://localhost:5173
Backend	http://localhost:8080
Mail inbox (Mailpit)	http://localhost:8025
Postgres	localhost:5434

No mail leaves the machine locally — Mailpit catches everything and shows it in a web inbox, so verification codes can be read without a real mail account.

Showing it to someone

1. EC2 Console → Instance state → **Start**
2. Wait about 90 seconds — DNS, containers and certificate handle themselves
3. Open <https://agromaster.duckdns.org>
4. **Stop the instance afterwards**

If something is wrong

```
sudo journalctl -u agromaster -n 50      # what the boot sequence did
docker compose ps                       # all three should be healthy
docker compose logs backend --tail 50
```

SYMPTOM	CAUSE
Site unreachable, instance running	DNS not updated — run <code>deploy/update-dns.sh</code>
Certificate warning	Expired while off; renew and reload
Backend unhealthy	Usually the Neon credentials
Google button missing	<code>GOOGLE_CLIENT_ID</code> empty in <code>.env</code>

11. Known gaps

Test coverage is the weakest part of this project, and it is better to say so directly than to be found out.

There is one backend test — a Spring context load proving every bean wires together — and no frontend tests. Everything else was verified by hand against a running system. CI runs that one test, so it catches compile errors and wiring mistakes but little else.

In priority order, the tests worth writing:

1. `VerificationService` — the attempt limit, single use, expiry. Security logic with no test is a liability.
2. Auth endpoint integration tests
3. The reminder scheduler's no-duplicate guarantee

4. Frontend: the registration flow

Also absent: no staging environment, a single application instance (the reminder scheduler assumes this), and no error monitoring beyond container logs.

12. Questions you are likely to be asked

"Walk me through what happens when someone logs in." The browser posts to `/api/auth/login`. The backend loads the user by email, compares the password with BCrypt, and returns a signed JWT with a 24-hour expiry. The browser stores it and attaches it to every subsequent request, where `JwtFilter` verifies the signature and loads the user before the controller runs. Both failure cases — unknown email and wrong password — return the same message, so the endpoint cannot be used to discover which addresses are registered.

"Why is the AI a separate service?" The AI work is Python and the business logic is Java, so separating them avoids forcing one language to do both. It also means the AI provider can change without touching the backend, and an AI outage cannot take down crop management. The cost is an extra network hop and another container.

"Your AI endpoint was public at one point. How did you find that?" A security review of the running system rather than the code. I tested the endpoints from outside and found `POST /ai/ask` returning 200 with no authorization header — an unmetered LLM endpoint billed to my API key. The fix was to route it through the authenticated backend, delete the nginx passthrough, and stop publishing the port.

"How do you stop someone reading another farmer's data?" Ownership is in the query, not a check afterwards. Repository methods are `findByIdAndUserId` and `findByIdAndCropUserId`, so a request for someone else's crop id returns nothing rather than returning a row that is then filtered. I checked for bare `findById` calls and the only ones pass the authenticated user's own id.

"A 6-digit code is only a million combinations. Isn't that weak?" On its own, yes — it is trivially brute-forceable. It is safe because of what surrounds it: 10-minute expiry, five attempts before the code dies, a 60-second resend cooldown, per-IP rate limiting, and single use. Remove the attempt limit and the whole thing becomes theatre.

"Tell me about a bug you found that was hard to spot." The attempt counter on verification codes. On a wrong guess the counter was incremented, saved, and then an exception thrown — but that exception extended `RuntimeException`, so `@Transactional` rolled the increment back. The brute-force limit silently did nothing while looking completely correct in the code. I caught it by checking the database after five wrong guesses and seeing zero. Fixed with `noRollbackFor`.

"Why build images in CI instead of on the server?" The instance has 1 GB of RAM and the Maven build was regularly OOM-killed — the reason a swapfile existed. Building in CI removed the swap requirement, cut deploys from five minutes to about thirty seconds, and made every deployed version traceable to a commit SHA, which gives a real rollback path.

"How would you deploy this properly, without SSH open?" AWS SSM Run Command — no inbound port 22 at all, with GitHub authenticating via IAM and OIDC instead of holding a private key. I did not build it because it

needs an instance role and an OIDC provider for a demo box that is powered off most of the time, but that is the correct answer for anything handling real data.

"What would you do next?" Tests, starting with `VerificationService`, because security logic without tests is the largest risk in the project. After that, crop disease diagnosis from a photograph — the AI provider is already multimodal, and it is the highest-value feature available for the actual users.

End of document.